

Estándares técnicos del squad

Entregable 4.3

Versión 1.0

Documento de referencia para el squad

ProviderOptimizer y componentes relacionados

Estándares técnicos del squad — Entregable 4.3

Versión: 1.0

Propósito: documento de referencia para el squad que define convenciones de código, políticas de trabajo y prácticas operativas que deben cumplirse en el desarrollo del microservicio ProviderOptimizer y componentes relacionados.

Lenguaje y alcance: .NET (C#), React (front), Docker y prácticas de squad (branching, code review, CI/CD, secrets, arquitectura base, deuda técnica).

1) Convenciones de código — .NET

- Estándares generales
- Target framework: .NET 10.0.
- Usar nullable reference types (enabled) y C# 12+ features cuando aporten claridad.
- Formato: 4 espacios por tabulador, máximo 120 columnas en líneas complejas.
- Archivo por tipo principal (una clase pública por fichero) salvo excepciones justificadas.
- Nombres
- PascalCase para tipos y métodos públicos; camelCase para parámetros y variables locales; `_camelCase` para campos privados inyectados si se desea (o `_logger`).
- Interfaces con prefijo `I` (`IRepository`, `IOptimizerService`).
- Principios de diseño
- Clean Architecture (capas: API, Application, Domain, Infrastructure). Las dependencias siempre apuntan hacia adentro.
- Respetar SOLID: inyección de dependencias, SRP, DIP (usar interfaces en Application e implementaciones en Infrastructure).
- Código y patrones
- Métodos asíncronos: `async/await` terminando en "Async".
- Evitar métodos estáticos con estado mutable.
- DTOs para entrada/salida de API; mapear con manual mapping o AutoMapper con perfil explícito.
- Repositorios/UnitOfWork en Infrastructure; Application no debe usar EF Core directamente.
- Tests
- Cobertura mínima para nuevos features: 70% (ideal >80% para módulos críticos).
- Tests unitarios con xUnit/NUnit; usar Moq o NSubstitute para mocks.
- Al menos 1 test de integración por flujo crítico (ej. Optimize end-to-end con InMemory o SQL de prueba).

2) Convenciones de código — React

- Estándares generales
- Usar TypeScript para nuevas UIs (si es viable). Si se usa JavaScript, seguir ES2020+.
- Estructura de carpetas: `components/`, `hooks/`, `pages/` (si aplica), `services/`, `styles/`.
- Componentes pequeños y componibles; dividir en Presentational / Container cuando sea necesario.

- Nombres y estilo
- PascalCase para componentes y archivos de componente (MyComponent.tsx).
- Hooks personalizados prefijados con use (useProviders).
- CSS Modules o styled-components para estilos; evitar estilos globales salvo tema/variables.
- Estado y efectos
- Preferir estado local para UI y React Query / SWR para fetching y caching; Redux solo si la complejidad lo exige.
- Manejar efectos con limpieza y abort controllers para peticiones fetch.
- Tests
- Testing library (React Testing Library) para componentes; Jest como runner.
- Tests de integración para flows críticos (login, solicitud de asistencia, seguimiento en tiempo real).

3) Convenciones de contenedores — Docker

- Dockerfiles
- Usar multi-stage builds: build con sdk, runtime con aspnet/appropriate runtime.
- Minimizar capas y limpiar paquetes temporales.
- No ejecutar procesos como root en contenedores de producción.
- Establecer HEALTHCHECK en imágenes para orquestadores.
- Imágenes y etiquetas
- Versionar imágenes: registry/project:semver (ej. asisya/provider-optimizer:1.0.0) y tag latest solo en CD controlado.
- Evitar secretos en imágenes; usar variables de entorno y vaults.
- docker-compose
- Usarlo para entorno local con servicios dependientes (DB, broker); no para producción.

4) Políticas del squad

- Branch strategy
- GitHub Flow / Trunk-based: rama main protegida y ramas de feature por issue (feature/ISS-123-descripcion).
- Todas las PRs deben ir a main vía Pull Request, con CI verde y reviews aprobados.
- Code reviews
- PR máximo recomendado: < 400 líneas; preferir PRs pequeños y atómicos.
- Checklist en PR: build passing, tests, documentación mínima del cambio, migraciones incluidas si aplica, changelog si impacta contrato.
- CI/CD
- Pipeline de PR: restore, build, lint, run unit tests, seguridad básica (dependency scan), publish test artifacts.
- Pipeline main: build, run integration tests, publish image to registry (staging tag), despliegue a staging, smoke tests, despliegue a producción con aprobación manual.
- Entorno de staging debe replicar producción (infra as code) y tener datos sanitizados.
- Definición de Done

- Código integrado en main y desplegado en staging.
- Tests unitarios y de integración relevantes pasan.
- Documentación mínima actualizada (README, API contract).
- Migraciones aplicadas o scripts de inicialización incluidos.
- Rollback plan identificado si cambio es riesgoso.
- Manejo de secretos
- Nunca almacenar secretos en repositorio (incluso en branches privados).
- Usar Secret Manager del cloud (AWS Secrets Manager, Azure Key Vault) o vault centralizado.
- CI: usar variables secretas del runner/registry; logs deben ocultar valores sensibles.
- Arquitectura base del squad
- Baseline: Clean Architecture con proyectos independientes: Api (exposición HTTP), Application (use cases), Domain (entidades), Infrastructure (EF, adaptadores externos).
- Comunicación entre servicios: eventos (pub/sub) y colas (SQS/SNS o equivalente). Contratos versionados (OpenAPI + mensajes con version header).
- Observabilidad: logs estructurados JSON, métricas (Prometheus/CloudWatch), trazas distribuidas (X-Ray/Jaeger).
- Resiliencia: circuit-breaker, retries con backoff (Polly), timeouts y bulkheads si aplica.
- Control de deuda técnica
- Mantener un backlog de deuda técnica con prioridad y estimación.
- Reservar al menos 10-20% del sprint para deuda técnica/estabilidad.
- Medir indicadores: code coverage, cyclomatic complexity, lint warnings, vulnerabilidades de dependencias.
- Revisiones trimestrales de arquitectura y tech radar para decidir retiros de frameworks/dep.

5) Herramientas recomendadas

- .NET: dotnet CLI, editorconfig, Roslyn analyzers (Microsoft.CodeAnalysis.FxCopAnalyzers o recommended), StyleCop si se desea.
- React: ESLint + Prettier, TypeScript, Storybook para componentes UI.
- Docker/CI: Docker Desktop (local), GitHub Actions / Azure DevOps / GitLab CI para pipelines.
- Seguridad: Snyk/Dependabot para dependencias, trivy para imágenes.
- Observabilidad: Grafana/Prometheus or CloudWatch + X-Ray.

6) Plantilla mínima de PR (incluir en cada PR)

- Título: [ISS-xxx] Resumen corto
- Descripción: problema resuelto, enfoque, pruebas realizadas, riesgos, pasos para QA.
- Checklist:
 - ☐ Build y tests locales pasan
 - ☐ Lint y analizadores OK
 - ☐ Documentación actualizada

- [] PR aprobada por al menos 1 revisor

7) Enmiendas y versionado del estándar

- Este documento es viva: cambios mayores deben versionarse (v1.0, v1.1) y aprobarse en la retro del squad.

Contacto: Standards owner — Tech Lead del squad. Para dudas o cambios abrir issue "standards" en el repo.